BCS703 – MODULE 4

Kerberos (Version 4 and 5, differences) • S/MIME • Pretty Good Privacy (PGP)

#	Topic
1	Kerberos – motivation, requirements, threats
2	Kerberos Version 4 (dialogues, message exchanges, realms)
3	Kerberos Version 5 (dialogue, flags)
4	Differences between Kerberos Version 4 and Version 5
5	S/MIME (services, content types, messages, algorithms, certificate processing)
6	Pretty Good Privacy (PGP / OpenPGP) and PGP vs S/MIME
7	Last-minute revision sheet

How to use: every topic has (1) a quick definition, (2) the steps/points in the order the textbook gives them, (3) the textbook figure, and (4) an **Exam keywords** box. Write the bold words in your answer to score marks. Content and figures are taken only from your uploaded textbook pages.

1. KERBEROS (Section 15.3)

Kerberos is an **authentication service** developed as part of **Project Athena at MIT**. Name from Greek mythology: the **three-headed dog guarding the entrance of Hades** – three components were intended: **authentication, accounting, audit** (the last two were never implemented).

Problem addressed: in an open distributed environment, users at workstations wish to access services on servers distributed through the network. Servers must **restrict access to authorized users** and **authenticate requests for service**. A workstation cannot be trusted to identify its users correctly. **Three threats:**

1. A user gains access to a workstation and **pretends to be another user** operating from that workstation.
2. A user **alters the network address** of a workstation so requests appear to come from the impersonated workstation.
3. A user **eavesdrops** on exchanges and uses a **replay attack** to gain entrance to a server or disrupt operations.

Instead of building elaborate authentication protocols at each server, Kerberos provides a **centralized authentication server** that authenticates users to servers and servers to users. **Kerberos relies exclusively on symmetric encryption** (no public-key encryption). It is a **trusted third-party authentication service** based on the **Needham–Schroeder** protocol. Two versions in use: **Version 4** and **Version 5** (corrects security deficiencies of v4; **RFC 4120, RFC 4121**).

1.1 Motivation – three approaches to security in a distributed architecture

1. Rely on each client workstation to assure the identity of its user and rely on each server to enforce security policy based on user ID.
2. Require client systems to authenticate themselves to servers, but trust the client system concerning the identity of its user.
3. Require the **user to prove identity for each service invoked** and require **servers to prove their identity to clients**. → **Kerberos supports this third approach** (needed in an open environment).

1.2 Requirements (first report, [STEI88])

Requirement	Meaning
Secure	A network eavesdropper should not be able to obtain information needed to impersonate a user; Kerberos should not be the weak link.
Reliable	Lack of availability of Kerberos means lack of availability of supported services; so use a **distributed server architecture** with one system able to back up another.
Transparent	User should not be aware that authentication is taking place beyond entering a password.
Scalable	Support large numbers of clients and servers → modular, distributed architecture.

2. KERBEROS VERSION 4

Uses **DES** in an elaborate protocol. Built up through hypothetical dialogues (Bill Bryant, Project Athena); each dialogue fixes a vulnerability of the previous one.

2.1 A simple authentication dialogue

An **authentication server (AS)** knows the passwords of all users (centralized database) and shares a **unique secret key with each server**.

```
(1) C → AS : IDc || Pc || IDv
(2) AS → C : Ticket
(3) C → V  : IDc || Ticket
Ticket = E(Kv, [IDc || ADc || IDv])
```

Symbol	Meaning
C / AS / V	client / authentication server / server
IDc, IDv	identifier of user on C / of V
Pc	password of user on C
ADc	network address of C
Kv	secret key shared by AS and V

- The **ticket is encrypted** to prevent alteration/forgery; **IDv** lets the server check it decrypted correctly; **IDc** shows the ticket was issued for C; **ADc** counters an opponent capturing the ticket and using it from another workstation.

Problems: (i) user must enter the password for **every new service/ticket**; (ii) the **password is sent in plaintext** in message (1) – an eavesdropper can capture it.

2.2 A more secure dialogue – introduces the Ticket-Granting Server (TGS)

```
Once per user logon session:
(1) C → AS  : IDc || IDtgs
(2) AS → C  : E(Kc, Ticket_tgs)
Once per type of service:
(3) C → TGS : IDc || IDv || Ticket_tgs
(4) TGS → C : Ticket_v
Once per service session:
(5) C → V   : IDc || Ticket_v
Ticket_tgs = E(Ktgs,[IDc||ADc||IDtgs||TS1||Lifetime1])
Ticket_v   = E(Kv,  [IDc||ADc||IDv ||TS2||Lifetime2])
```

- Client asks AS for a **ticket-granting ticket**; the AS reply is encrypted with **Kc derived from the user's password** – the password is never sent; only the right user can decrypt.
- **Timestamp + lifetime** in the ticket stop an opponent reusing a stolen ticket later.
- The client then asks the TGS for a **service-granting ticket** for each service; tickets are **reusable**, so only **one password query per session**.

Remaining problems: (i) lifetime of ticket-granting ticket – too short means repeated password prompts, too long gives opponent a replay chance; a service must prove that the person using a ticket is the **same person to whom it was issued**; (ii) **servers must authenticate themselves to users** (else a false server could capture information).

2.3 The Version 4 authentication dialogue (Table 15.1, Fig 15.2, Fig 15.3)

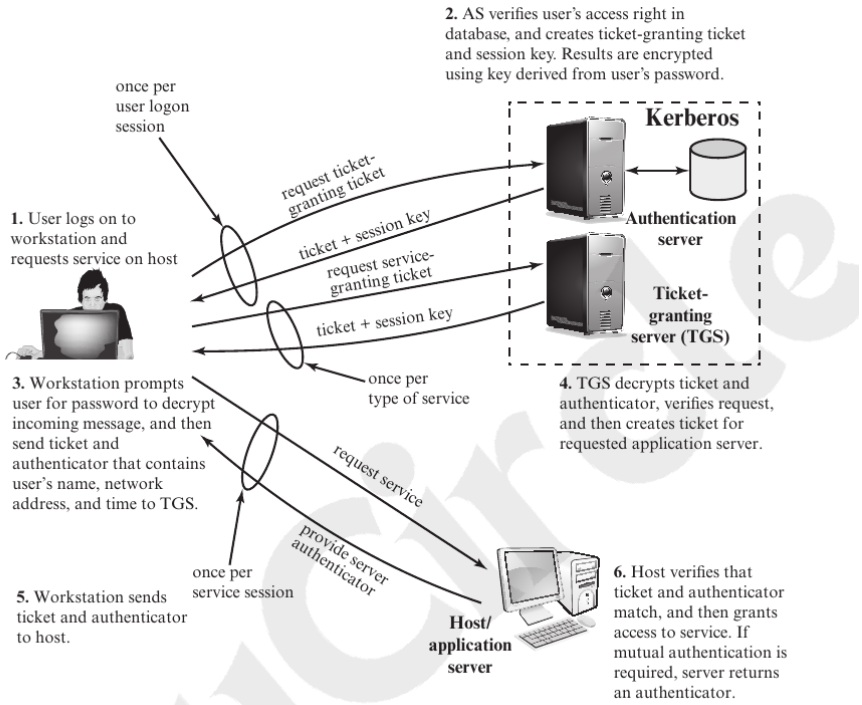


Figure 15.2 Overview of Kerberos

Fig 15.2 Overview of Kerberos

Phase	Message	Contents
(a) **Authentication Service exchange** – once per user logon session, to obtain ticket-granting ticket	(1) C → AS	$ID_c \| \| ID_{tgs} \| \| TS_1$
	(2) AS → C	$E(K_c, [K_{c,tgs} \| \| ID_{tgs} \| \| TS_2 \| \| Lifetime_2 \| \| Ticket_{tgs}])$ $Ticket_{tgs} = E(K_{tgs}, [K_{c,tgs} \| \| ID_C \| \| AD_C \| \| ID_{tgs} \| \| TS_2 \| \| Lifetime_2])$

Phase	Message	Contents
(b) **Ticket-Granting Service exchange** – once per type of service, to obtain service-granting ticket	(3) C → TGS	ID_V || $Ticket_{tgs}$ || $Authenticator_C$
	(4) TGS → C	$E(K_{c,tgs}, [K_{c,v} \| ID_v \| TS_4 \| Ticket_v])$ $Ticket_v = E(K_v, [K_{c,v} \| ID_C \| AD_C \| ID_V \| TS_4 \| Lifetime_4])$ $Authenticator_c = E(K_{c,tgs}, [ID_C \| AD_C \| TS_3])$
(c) **Client/Server Authentication exchange** – once per service session, to obtain service	(5) C → V	$Ticket_v$ || $Authenticator_C$ ($Authenticator_c = E(K_{c,v}, [ID_C \| AD_C \| TS_5])$)
	(6) V → C	$E(K_{c,v}, [TS_5 + 1])$ (for **mutual authentication**)

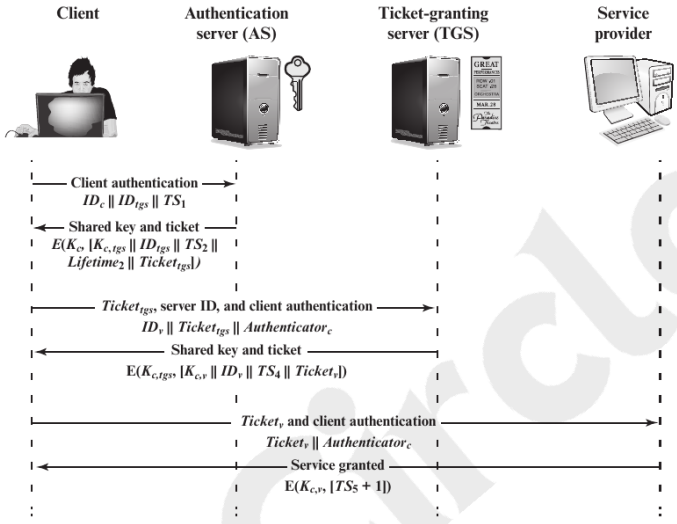


Figure 15.3 Kerberos Exchanges

Fig 15.3 Kerberos Exchanges

Explanation of each step (Table 15.2 – rationale)

1. **Msg 1:** client requests ticket-granting ticket. IDc tells AS the user; IDtgs says user wants the TGS; **TS1** lets AS verify the client's clock is synchronized.
2. **Msg 2:** AS returns ticket-granting ticket, encrypted with **Kc (from user's password)** – lets AS and client verify password and protects contents. $\mathbf{K_{c,tgs}}$ = **session key** created by AS so client and TGS can talk securely without a permanent shared key. TS2 = time issued; Lifetime2 = validity. $Ticket_{tgs}$ is encrypted with $\mathbf{K_{tgs}}$ (known only to AS and TGS) to prevent tampering.
3. **Msg 3:** client requests service-granting ticket: IDv (server wanted), $Ticket_{tgs}$ (assures TGS the user was authenticated by AS), and $\mathbf{Authenticator_c}$ generated by client to validate the ticket.
4. **Msg 4:** TGS returns service-granting ticket encrypted with $\mathbf{K_{c,tgs}}$; includes $\mathbf{K_{c,v}}$ – session key for client and server V. $Ticket_v$ encrypted with K_v (known only to TGS and server).
5. **Msg 5:** client sends $Ticket_v$ + $Authenticator_c$ to the server. Server decrypts the ticket, recovers $K_{c,v}$, decrypts the authenticator and matches ID and address.
6. **Msg 6:** optional – server returns **TS5 + 1** encrypted with $K_{c,v}$: proves the server is genuine (only V could create it) and is **not a replay**.

- **Ticket vs Authenticator:** the ticket is **reusable** and only **distributes keys securely** ("anyone who uses $K_{c,tgs}$ must be C"); the **authenticator proves identity**: used only **once**, very **short lifetime** ("at time TS3 I hereby use $K_{c,tgs}$"). So a stolen ticket + authenticator cannot be replayed later.
- At the end, client and server **share a secret key** for protecting future messages.

2.4 Kerberos realms and multiple Kerberi

A full-service Kerberos environment needs: (1) the Kerberos server has the **user ID and hashed passwords** of all users – all users registered; (2) the Kerberos server **shares a secret key with each server** – all servers registered. Such an environment is a **Kerberos realm**.

- **Realm:** a set of managed nodes that share the **same Kerberos database**. The database resides on the **Kerberos master computer** (physically secure room); a **read-only copy** may be on other Kerberos systems; all

changes are made only on the master (needs the master password).

- **Principal:** a service or user known to Kerberos, identified by **principal name** = **service/user name + instance name + realm name**.
- **Interrealm authentication** needs a **third requirement**: the Kerberos server of each interoperating realm **shares a secret key with the server of the other realm** (the two Kerberos servers are registered with each other), and each realm must **trust** the other's Kerberos server.

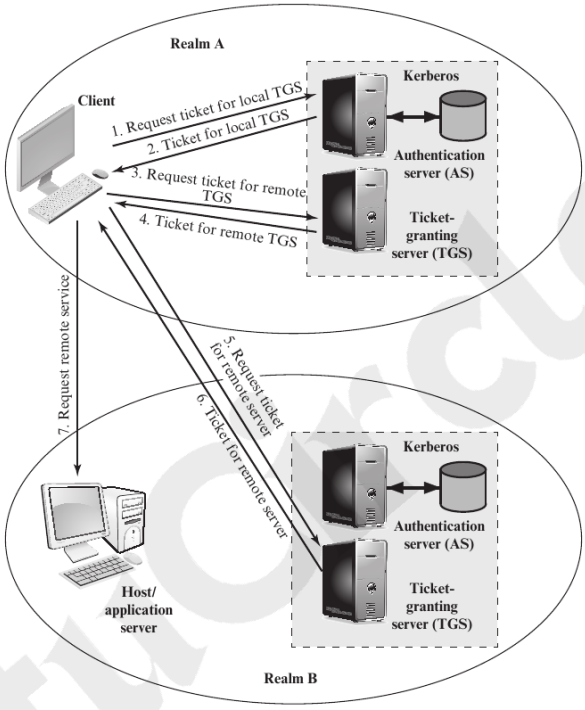


Figure 15.4 Request for Service in Another Realm

Fig 15.4 Request for Service in Another Realm

```
(1) C → AS     : IDc || IDtgs || TS1
(2) AS → C     : E(Kc,[Kc,tgs||IDtgs||TS2||Lifetime2||Ticket_tgs])
(3) C → TGS    : IDtgsrem || Ticket_tgs || Authenticator_c
(4) TGS → C    : E(Kc,tgs,[Kc,tgsrem||IDtgsrem||TS4||Ticket_tgsrem])
(5) C → TGSrem : IDvrem || Ticket_tgsrem || Authenticator_c
(6) TGSrem → C : E(Kc,tgsrem,[Kc,vrem||IDvrem||TS6||Ticket_vrem])
(7) C → Vrem   : Ticket_vrem || Authenticator_c
```

- Client gets a ticket for the **local TGS**, then asks it for a ticket-granting ticket for the **remote TGS**, then asks the remote TGS for a ticket for the remote server. The remote server decides whether to honour the request (ticket shows the realm where user was authenticated).
- **Drawback:** does not scale – with **N realms**, **N(N−1)/2** secure key exchanges are needed.

Exam keywords – Kerberos v4
Project Athena • authentication server (AS) • ticket-granting server (TGS) • ticket-granting ticket • service-granting ticket • session key • authenticator • timestamp • lifetime • nonce • mutual authentication (TS5+1) • DES • PCBC • realm • principal • interrealm authentication • N(N−1)/2 • symmetric encryption only • trusted third party • Needham–Schroeder

3. KERBEROS VERSION 5 (RFC 4120)

Version 5 improves version 4 in two areas: **environmental shortcomings** and **technical deficiencies** ([KOHL94], [BELL90]).

3.1 The Version 5 authentication dialogue (Table 15.3)

Phase	Message	Contents
(a) Authentication Service exchange (ticket-granting ticket)	(1) C → AS	$Options \| ID_c \| Realm_c \| ID_{tgs} \| Times \| Nonce_1$
	(2) AS → C	$Realm_C \| ID_C \| Ticket_{tgs} \| E(K_c, [K_{c,tgs} \| Times \| Nonce_1 \| Realm_{tgs} \| ID_{tgs}])$ $Ticket_{tgs} = E(K_{tgs}, [Flags \| K_{c,tgs} \| Realm_c \| ID_C \| AD_C \| Times])$
(b) Ticket-Granting Service exchange (service-granting ticket)	(3) C → TGS	$Options \| ID_v \| Times \| Nonce_2 \| Ticket_{tgs} \| Authenticator_c$
	(4) TGS → C	$Realm_c \| ID_C \| Ticket_v \| E(K_{c,tgs}, [K_{c,v} \| Times \| Nonce_2 \| Realm_v \| ID_v])$ $Ticket_v = E(K_v, [Flags \| K_{c,v} \| Realm_c \| ID_C \| AD_C \| Times])$; $Authenticator_c = E(K_{c,tgs}, [ID_C \| Realm_c \| TS_1])$
(c) Client/Server Authentication exchange (service)	(5) C → V	$Options \| Ticket_v \| Authenticator_c$
	(6) V → C	$E(K_{c,v}, [TS_2 \| Subkey \| Seq\ \#])$ $(Authenticator_c = E(K_{c,v}, [ID_C \| Realm_c \| TS_2 \| Subkey \| Seq\ \#]))$

New elements in version 5

- **Realm** – realm of the user. **Options** – request that certain **flags** be set in the returned ticket.
- **Times** – client requests time settings: **from** (desired start time), **till** (requested expiration), **rtime** (requested renew-till time).
- **Nonce** – random value repeated in message (2) to assure the response is **fresh** and not replayed.
- In message (5)/(6): **Subkey** – client's choice of encryption key to protect this specific application session (if omitted, the session key $K_{c,v}$ is used); **Sequence number** – optional starting sequence number for messages (to detect replays).
- Message (6): server returns the timestamp from the authenticator; in v5 it is **not incremented by one** because message format makes it impossible for an opponent to create message (6) without the keys.
- Authenticator essentially the same as in v4; mutual authentication can be requested as an **option**.

3.2 Ticket flags (Table 15.4)

Flag	Meaning
INITIAL	Ticket issued using the AS protocol, not based on a ticket-granting ticket (e.g., password-changing server wants to know password was recently tested).
PRE-AUTHENT	During initial authentication the client was **authenticated by the KDC before a ticket was issued** (e.g., MIT: encrypted timestamp preauthentication).
HW-AUTHENT	Initial authentication needed **hardware** expected to be possessed solely by the named client (e.g., smart card).
RENEWABLE	Ticket can be used to obtain a **replacement ticket** that expires later; has two expiry times (this ticket + latest permissible). TGS may refuse to renew a reported-stolen ticket.
MAY-POSTDATE	TGS may issue a **postdated** ticket based on this ticket-granting ticket.
POSTDATED	Ticket has been postdated; end server can check *authtime* to see when the original authentication occurred (useful for long batch jobs).
INVALID	Ticket is invalid and must be **validated by the KDC** before use.
PROXIABLE	TGS may issue a new service-granting ticket with a **different network address**.
PROXY	This ticket is a proxy.
FORWARDABLE	TGS may issue a new ticket-granting ticket with a **different network address**.
FORWARDED	Ticket has been forwarded or was issued from a forwarded ticket-granting ticket.

Exam keywords – Kerberos v5
RFC 4120 • options • realm • times (from, till, rtime) • nonce • subkey • sequence number • flags • pre-authentication • renewable • postdated • proxiable • forwardable • ASN.1 / BER • CBC + checksum • any encryption type

4. DIFFERENCES BETWEEN KERBEROS VERSION 4 AND VERSION 5

4.1 Environmental shortcomings of v4 (and v5 improvements)

#	Aspect	Version 4	Version 5
1	**Encryption system dependence**	Requires **DES** (export restrictions, doubts about DES strength).	Ciphertext is tagged with an **encryption-type identifier** so **any encryption technique** can be used; keys tagged with type and length.
2	**Internet protocol dependence**	Requires **IP addresses**; other address types (e.g., ISO) not accommodated.	Network addresses tagged with **type and length**, any address type allowed.
3	**Message byte ordering**	Sender uses its **own byte ordering** and tags the message (not established convention).	All structures defined with **ASN.1 and BER** – unambiguous byte ordering.
4	**Ticket lifetime**	8-bit value in units of **5 minutes**: max $2^8 \times 5$ = **1280 minutes** (just over 21 hours).	Tickets include explicit **start time and end time** – **arbitrary lifetimes**.
5	**Authentication forwarding**	Credentials issued to one client **cannot be forwarded** to another host.	**Supported** – a server can access another server on behalf of the client (e.g., print server fetching client's file).
6	**Interrealm authentication**	N realms need about **N^2** Kerberos-to-Kerberos relationships.	Method needing **fewer relationships** (hierarchical, forwarding of TGTs).

4.2 Technical deficiencies of v4 (and v5 improvements)

#	Aspect	Version 4	Version 5
1	**Double encryption**	Tickets in messages (2) and (4) are **encrypted twice** (server key, then client key) – wasteful.	Tickets are **not double-encrypted**.
2	**PCBC encryption**	Non-standard **propagating CBC** mode of DES – vulnerable to ciphertext-block interchange attack.	**Standard CBC** with **explicit integrity mechanism** (checksum/hash attached before encryption).
3	**Session keys**	Same ticket reused → risk of replay of messages from an old session.	Client and server can negotiate a **subkey** for one connection only.
4	**Password attacks**	Vulnerable (AS reply encrypted with password-based key can be guessed offline).	Still vulnerable, but **pre-authentication** makes it harder (does not prevent it).

Other differences in the dialogue

- v5 adds **Options, Realm, Times, Nonce, Flags, Subkey, Sequence number**; v4 uses **TS + Lifetime** only.
- v4 mutual authentication returns **TS5+1**; v5 returns the timestamp (no increment needed).
- v5 allows the client to get a **service-granting ticket directly from the AS** (INITIAL flag); v4 only via TGS.
- v5 supports **renewable, postdated, proxy and forwarded tickets**; v4 does not.

5. S/MIME – Secure/Multipurpose Internet Mail Extension (Section 19.4)

S/MIME is a **security enhancement to the MIME Internet email format** standard, based on technology from **RSA Data Security**. Key documents: **RFC 5750** (S/MIME v3.2 certificate handling), **RFC 5751** (v3.2 message specification), RFC 4134 (examples), **RFC 2634** (enhanced security services), **RFC 5652** (Cryptographic Message Syntax, CMS), RFC 3370 (CMS algorithms), RFC 5752 (multiple signatures), RFC 1847 (multipart/signed and multipart/encrypted).

Email threats: authenticity-related, integrity-related, confidentiality-related, availability-related (NIST SP 800-177). S/MIME provides **authentication, integrity, non-repudiation (digital signatures) and confidentiality (encryption)** of the message body.

5.1 Four services (Table 19.4)

Function	Typical algorithm	Typical action
Digital signature	RSA / SHA-256	Hash code of message created using SHA-256; the digest is encrypted with the **sender's private key** and included with the message.
Message encryption	AES-128 with CBC	Message encrypted with a **one-time session key** generated by the sender; session key encrypted with **RSA using recipient's public key** and included.
Compression	unspecified	Message may be compressed for storage or transmission.
Email compatibility	Radix-64 conversion	Encrypted message converted to an **ASCII string** using radix-64 for transparency to email applications.

5.2 Authentication (digital signature)

1. Sender creates a message.
2. **SHA-256** generates a **256-bit message digest**.
3. Digest is encrypted with **RSA using sender's private key**; result appended to the message with identifying information of the signer (to retrieve the public key).
4. Receiver uses RSA with the **sender's public key** to decrypt and recover the digest.
5. Receiver generates a new digest of the message and **compares** it with the decrypted hash; if equal → **authentic**.

- **Detached signatures** are supported – stored/transmitted separately from the message. Useful for: a **signature log** of messages; detecting **virus infection** of an executable; documents signed by **more than one party** (e.g., legal contract) – each signature independent, otherwise signatures would have to be nested.

5.3 Confidentiality

Most commonly **AES with 128-bit key in CBC mode**; the key is encrypted with RSA. Each symmetric key is a **content-encryption key used only once** (new random number for each message), bound to the message and sent with it.

1. Sender generates the message and a **random 128-bit number** = content-encryption key for this message only.
2. Message is encrypted with the content-encryption key.
3. Content-encryption key is encrypted with **RSA using the recipient's public key** and attached.
4. Receiver uses RSA with its **private key** to recover the content-encryption key.
5. Content-encryption key decrypts the message.

- Symmetric + public-key combination is used because symmetric algorithms are **much faster** for large content; public-key solves **key-distribution** (no session-key exchange protocol needed – each mail is a one-time event, and email is store-and-forward so handshaking is impractical); **one-time keys** strengthen security (only a little plaintext per key, no relationship among keys).

5.4 Confidentiality and authentication together (Fig 19.5)

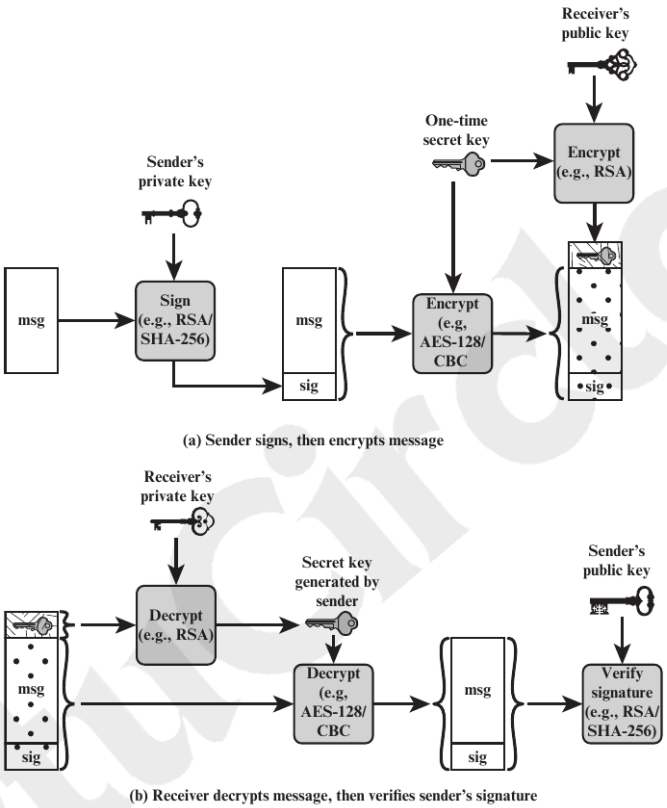


Figure 19.5 Simplified S/MIME Functional Flow

Fig 19.5 Simplified S/MIME Functional Flow

- (a) **Sender signs, then encrypts**: signature on plaintext message appended; message + signature encrypted as one block with symmetric key (e.g., AES-128/CBC); symmetric key encrypted with receiver's public key (RSA). (b) **Receiver decrypts** (RSA private key → secret key → decrypt message) **then verifies** signature with sender's public key.
- Order may be either way. **Sign first:** signer's identity hidden by encryption; convenient to store signature with plaintext; third party need not know the symmetric key. **Encrypt first:** signature can be verified without exposing content (no private key needed), but recipient cannot relate signer and plaintext.

5.5 Email compatibility and compression

- Many email systems allow only **ASCII text**. S/MIME converts the raw 8-bit binary stream into printable ASCII (**7-bit encoding**) using **Base64**: **each group of three octets → four ASCII characters**. Base64 converts blindly, so even a signed-only ASCII message becomes unreadable to the casual observer.
- **Compression** saves space for transmission and storage; can be applied in any order with signing/encryption. Compressing binary encrypted data is discouraged (no gain); Base64 encrypted data may benefit; if **lossy compression** is used with signing → **compress first, then sign**.

5.6 S/MIME message content types (CMS, RFC 5652)

Type	Use
Data	Inner MIME-encoded message content; may be encapsulated in SignedData, EnvelopedData or CompressedData. Also used for **clear signing**.
SignedData	Apply a **digital signature** to a message.
EnvelopedData	**Encrypted content** of any type + encrypted content-encryption keys for one or more recipients.
CompressedData	Apply **data compression** to a message.

5.7 Cryptographic algorithms (Table 19.5)

Function	Requirement
Create message digest for signature	**MUST** support **SHA-256**; SHOULD support SHA-1; receiver SHOULD support MD5 (backward compatibility)
Form digital signature	**MUST** support **RSA with SHA-256**; SHOULD support DSA with SHA-256, RSASSA-PSS with SHA-256, RSA with SHA-1, DSA with SHA-1, RSA with MD5
Encrypt session key	**MUST** support **RSA encryption**; SHOULD support RSAES-OAEP, Diffie–Hellman ephemeral-static mode
Encrypt message with one-time session key	**MUST** support **AES-128 with CBC**; SHOULD support AES-192 CBC, AES-256 CBC, Triple DES CBC

MUST = absolute requirement; **SHOULD** = valid reasons may exist to ignore, but recommended (RFC 2119).
Choosing the content-encryption algorithm (sending agent rules, in order):

1. If recipient's preferred decrypting-capability list is known, use the **first (highest preference)** one the sender can use.
2. Else, if messages were received from the recipient, use the **same algorithm as the last signed and encrypted message** received.
3. Else, if no knowledge and willing to risk → **triple DES**.
4. Else, if no knowledge and not willing to risk → **RC2/40**.

5.8 S/MIME messages (PKCS objects)

All new application types use the designation **PKCS** (public-key cryptography specifications from RSA Laboratories). S/MIME secures a **MIME entity** (whole message except RFC 5322 headers, or a sub-part of multipart) with signature, encryption or both; the entity + security data (algorithm identifiers, certificates) produce a **PKCS object**, which is wrapped in MIME. Content is converted to **canonical form**.

(a) EnvelopedData (application/pkcs7-mime; smime-type=enveloped-data)

1. Generate a **pseudorandom session key** for a symmetric algorithm (RC2/40 or triple DES).
2. For each recipient, encrypt the session key with the recipient's **public RSA key**.
3. For each recipient prepare a **RecipientInfo** block: identifier of the recipient's public-key certificate, identifier of the algorithm used to encrypt the session key, and the encrypted session key.
4. Encrypt the message content with the session key.

RecipientInfo blocks + encrypted content = **envelopedData** → **base64**. Recipient: strips base64, uses private key to recover session key, decrypts content. Object is in **BER** (binary), so transfer-encode with base64.

(b) SignedData (smime-type=signed-data)

1. Select a **message digest algorithm** (SHA or MD5).
2. Compute the message digest of the content.
3. Encrypt the digest with the **signer's private key**.
4. Prepare **SignerInfo**: signer's public-key certificate, digest algorithm identifier, algorithm used to encrypt the digest, and the encrypted digest.

signedData = digest algorithm identifier + message + SignerInfo (+ optionally a set of certificates forming a chain from a root CA) → base64. Receiver decrypts digest with signer's public key, computes its own digest and compares.

(c) Clear signing (multipart/signed)

- Message is sent “in the clear”; recipients with MIME but **no S/MIME** can still read it. Two parts: **part 1** = message (any MIME type, 7-bit or base64/quoted-printable so it is not altered); **part 2** = **detached signature** (signedData with empty content) of type **application/pkcs7-signature**, base64 encoded.
- **protocol** parameter = two-part clear-signed entity; **micalg** = message digest type.

(d) Registration request and (e) Certificates-only message

- **application/pkcs10** carries a **certification request** to a CA: certificationRequestInfo (subject name + public key bit string), public-key algorithm identifier, and the signature of the block with the sender's private key.
- **Certificates-only (degenerate)**: application/pkcs7-mime with smime-type = degenerate; same as signedData but **no content and empty signerInfo**; carries certificates or a CRL in reply to a registration request.

5.9 S/MIME certificate processing

Uses **X.509 version 3** certificates. Each client is configured with a list of **trusted keys** and **CRLs** (local responsibility); certificates are signed by CAs. **User agent role – key management functions:**

- **Key generation:** generate separate **Diffie–Hellman and DSS** key pairs (should also generate RSA pairs) from a good non-deterministic random source; RSA length 768–1024 bits recommended, **never below 512**.
- **Registration:** public key registered with a CA to get an X.509 certificate.
- **Certificate storage and retrieval:** local list of certificates to verify incoming signatures and encrypt outgoing messages.

5.10 Enhanced security services (RFC 2634)

- **Signed receipts** – proof of delivery; recipient signs the original message plus the original signature and appends a new signature.
- **Security labels** – information on sensitivity of content (used for access control, priority such as secret/confidential/restricted, or role-based access).
- **Secure mailing lists** – a **Mail List Agent (MLA)** does the per-recipient encryption for one incoming message and forwards it.
- **Signing certificates** – securely bind the sender's certificate to the signature.

Exam keywords – S/MIME
MIME enhancement • RSA Data Security • digital signature RSA/SHA-256 • AES-128 CBC • content-encryption key (one-time) • compression • Base64 (3 octets → 4 characters) • detached signature • SignedData • EnvelopedData • CompressedData • clear signing multipart/signed • pkcs7-mime • RecipientInfo • SignerInfo • X.509 v3 • MUST / SHOULD • signed receipts • security labels • mail list agent

6. PRETTY GOOD PRIVACY – PGP (Section 19.5)

- PGP is an **alternative email security protocol** with **essentially the same functionality as S/MIME**.
- Created by **Phil Zimmerman**; first released as a product in **1991**; **free of charge** and popular for **personal use**. The initial protocol was **proprietary** and used some encryption algorithms with intellectual-property restrictions.
- **1996:** version 5.x defined in **RFC 1991 (PGP Message Exchange Formats)**. **OpenPGP** was then developed as a new standard based on PGP 5.x: **RFC 4880 (OpenPGP Message Format, Nov 2007)** and **RFC 3156 (MIME Security with OpenPGP, Aug 2001)**.

6.1 Two significant differences between S/MIME and OpenPGP

	S/MIME	OpenPGP / PGP
Key certification	Uses **X.509 certificates** issued by **Certificate Authorities** (or local agencies delegated by a CA). Trusted if there is a **valid PKIX chain to a trusted root**.	Users **generate their own** public/private keys and ask individuals/organizations that know them to **sign** their public key. A key is trusted if signed by another OpenPGP key trusted by the recipient → **Web of Trust**.
Key distribution	Sender's certificate/public key can accompany the message (signedData includes certificates).	Does **not include the sender's public key** with each message; recipient must **separately obtain** it. Organizations post keys on **TLS-protected websites**; keys may be registered with **OpenPGP public-key servers** (database organized by email address). **Anyone may post a key** with any email address – **no vetting**, so users rely on the Web of Trust.

- **NIST 800-177 recommends S/MIME rather than PGP** because of greater confidence in the **CA system** of verifying public keys.

Note: the textbook gives the detailed PGP description (services: authentication, confidentiality, compression, email compatibility, segmentation; key rings; message format) in **Appendix P**, which is **not part of the pages you uploaded**. If your syllabus asks for PGP operation in detail, upload Appendix P (or its PDF pages) and I will add a complete PGP section with its diagrams.

Exam keywords – PGP
Phil Zimmerman • 1991 • free • RFC 1991 • OpenPGP RFC 4880 • RFC 3156 • Web of Trust • users generate own keys • key servers • no vetting • S/MIME vs PGP • X.509 vs signed keys • NIST 800-177 prefers S/MIME

LAST-MINUTE REVISION SHEET – MODULE 4

Topic	One-line recall
Kerberos	Trusted third-party authentication service, MIT Project Athena, symmetric only; AS + TGS + server; tickets + authenticators; 4 requirements: secure, reliable, transparent, scalable.
v4 flow	(1) IDc||IDtgs||TS1 → (2) E(Kc,[Kc,tgs…Tickettgs]) → (3) IDv||Tickettgs||Authc → (4) E(Kc,tgs,[Kc,v…Ticketv]) → (5) Ticketv||Authc → (6) E(Kc,v,[TS5+1]).
Realm	Same Kerberos database; inter-realm = shared key between Kerberos servers; N(N−1)/2 exchanges; 7 messages via remote TGS.
v5 vs v4	Any cipher (v4: DES) • any address type • ASN.1/BER • explicit start/end times (v4: 1280 min) • forwarding • fewer realm relationships • no double encryption • CBC+checksum (v4: PCBC) • subkey • preauthentication.
S/MIME	Services: signature, encryption, compression, email compatibility (Base64). Types: Data, SignedData, EnvelopedData, CompressedData. Algorithms: RSA/SHA-256, AES-128 CBC. Clear signing = multipart/signed + pkcs7-signature.
PGP	Zimmerman 1991, RFC 1991 → OpenPGP RFC 4880; Web of Trust; keys from key servers; S/MIME preferred by NIST.